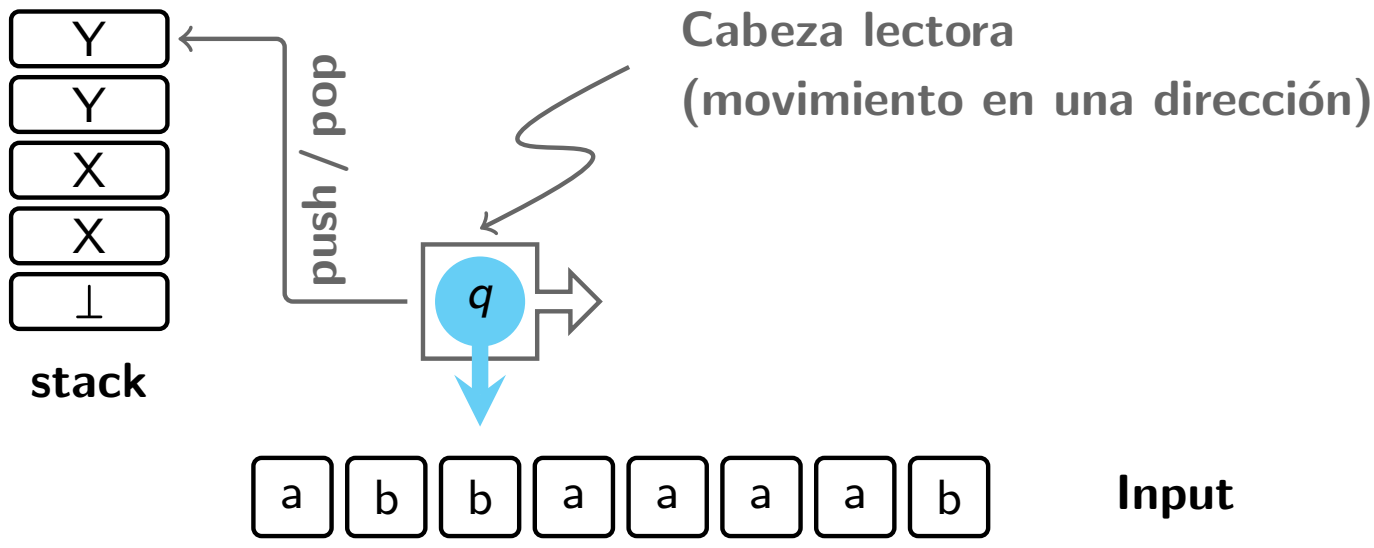
Gramáticas versus apiladores

Clase 24

IIC2223 / IIC2224

Prof. Cristian Riveros

Recordatorio: Autómatas apiladores



Recordatorio: Autómatas apiladores

Definición

Un **autómata apilador** (PushDown Automata, PDA) es una estructura:

$$\mathcal{P} \quad = \quad (Q, \Sigma, \Gamma, \Delta, q_0, \bot, F)$$

- Q es un conjunto finito de **estados**.
- Σ es el alfabeto de **input**.
- $q_0 \in Q$ es el estado **inicial**.
- F es el conjunto de estados **finales**.

+

- Γ es el alfabeto de **stack**.
- $\bot \in \Gamma$ es el símbolo **inicial de stack**.
- $\Delta \subseteq \big(Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma\big) \times \big(Q \times \Gamma^*\big)$ es una relación finita de transición.

Recordatorio: Autómatas apiladores

Definición

Un **autómata apilador** (PushDown Automata, PDA) es una estructura:

$$\mathcal{P} \quad = \quad (Q, \Sigma, \Gamma, \Delta, q_0, \bot, F)$$

Intuitivamente, la transición:

$$\big((p, a, A), (q, B_1 B_2 \cdots B_k) \big) \in \Delta$$

si el autómata apilador está:

- en el estado **p**,
- leyendo **a**, y
- en el tope del stack hay una **A**

entonces:

- cambia al estado **q**, y
- modifico el tope **A** por $\mathbf{B_1 B_2 \cdots B_k}$.

Recordatorio: Configuración de un autómata apilador

Sea $\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \perp, F)$ un autómata apilador.

Notación

Dado una palabra $A_1 A_2 \dots A_k \in \Gamma^+$ decimos que:

- $A_1 A_2 \dots A_k$ es un stack (contenido),
- A_1 es el **tope** del stack y
- $A_2 \dots A_k$ es la **cola** del stack.

Definición

Una **configuración** de $\mathcal{P}$ es una tupla $(q \cdot \gamma, w) \in (Q \cdot \Gamma^*, \Sigma^*)$ tal que:

- q es el estado actual.
- γ es el contenido del stack.
- w es el contenido del input.

Recordatorio: Configuración de un autómata apilador

Sea $\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \perp, F)$ un autómata apilador.

Definición

Decimos que una configuración:

$$(q \cdot \gamma, w) \quad \in \quad (Q \cdot \Gamma^*, \Sigma^*)$$

- es **inicial** si $q \cdot \gamma = q_0 \cdot \perp$.
- es **final** si $q \cdot \gamma = q_f \cdot \epsilon$ con $q_f \in F$ y $w = \epsilon$.

Recordatorio: Ejecución de un autómata apilador

Sea $\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \bot, F)$ un autómata apilador.

Definición

Se define la relación $\vdash_{\mathcal{P}}$ de **siguiente-paso** entre configuraciones de $\mathcal{P}$:

$$(q_1 \cdot \gamma_1, w_1) \vdash_{\mathcal{P}} (q_2 \cdot \gamma_2, w_2)$$

si, y solo si, existe una transición $(q_1, a, A, q_2, \alpha) \in \Delta$ y $\gamma \in \Gamma^*$ tal que:

- $w_1 = a \cdot w_2$
- $\gamma_1 = A \cdot \gamma$
- $\gamma_2 = \alpha \cdot \gamma$.

Se define $\vdash_{\mathcal{P}}^*$ como la clausura **refleja** y **transitiva** de $\vdash_{\mathcal{P}}$.

$(q_1\gamma_1, w_1) \vdash_{\mathcal{P}}^* (q_2\gamma_2, w_2)$ si uno puede ir de $(q_1\gamma_1, w_1)$ a $(q_2\gamma_2, w_2)$ en **0 o más pasos**.

Recordatorio: Lenguajes de un autómata apilador

Sea $\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \perp, F)$ un autómata apilador y $w \in \Sigma^*$.

Definiciones

- $\mathcal{P}$ **acepta** w si, y solo si, $(q_0 \perp, w) \vdash_{\mathcal{P}}^{*} (q_f, \epsilon)$ para algún $q_f \in F$.
- El **lenguaje** aceptado por $\mathcal{P}$ se define como:

$$\mathcal{L}(\mathcal{P}) \quad = \quad \{w \in \Sigma^* \mid \mathcal{P} \text{ acepta } w\}$$

Outline

Apiladores alternativos (anterior)

Desde CFG a PDA

Outline

Apiladores alternativos (anterior)

Desde CFG a PDA

Autómatas apiladores alternativos

Veremos otra definición **alternativa** y
poco común de autómatas apiladores.

¿por qué?

1. Este modelo nos ayudara a entender mejor
   los algoritmos de evaluación para gramáticas.
2. Modelo menos estándar pero mucho más sencillo.
3. El profesor le gusta y lo encuentra interesante :).

Autómatas apiladores alternativos

Definición

Un **PDA alternativo** es una estructura:

$$\mathcal{D} \quad = \quad (Q, \Sigma, \Delta, q_0, F)$$

- Q es un conjunto finito de **estados**.
- Σ es el alfabeto de **input**.
- $q_0 \in Q$ es el estado **inicial**.
- F es el conjunto de estados **finales**.

$+$

- $\Delta \subseteq Q^+ \times (\Sigma \cup \{\epsilon\}) \times Q^*$ es una **relación finita de transición**.

Autómatas apiladores alternativos

Definición

Un **PDA alternativo** es una estructura:

$$\mathcal{D} \quad = \quad (Q, \Sigma, \Delta, q_0, F)$$

Intuitivamente, la transición:

$$\left(A_1 \cdots A_i,\ a\ , B_1 \cdots B_j \right) \ \in \ \Delta$$

si el autómata apilador tiene:

- $A_1 \ldots A_i$ en el tope del stack y
- leyendo **a**

entonces:

- cambia el tope $A_1 \ldots A_i$ por $B_1 \ldots B_j$.

No hay diferencia entre estados y alfabeto del stack.

Configuración de un autómata apilador alternativo

Sea $\mathcal{D} = (Q, \Sigma, \Delta, q_0, F)$ un PDA alternativo.

Definición

Una **configuración** de $\mathcal{D}$ es una tupla $(q_1 \dots q_k, w) \in (Q^+, \Sigma^*)$ tal que:

- $q_1 \dots q_k$ es el contenido del stack con q_1 el tope del stack.
- w es el contenido del input.

Decimos que una configuración:

- (q_0, w) es **inicial**.
- (q_f, ϵ) es **final** si $q_f \in F$.

Ejecución de un autómata apilador alternativo

Sea $\mathcal{D} = (Q, \Sigma, \Delta, q_0, F)$ un PDA alternativo.

Definición

Se define la relación $\vdash_{\mathcal{D}}$ de **siguiente-paso** entre configuraciones de $\mathcal{D}$:

$$(\gamma_1, w_1) \vdash_{\mathcal{D}} (\gamma_2, w_2)$$

si, y solo si, existe una transición $(\alpha, a, \beta) \in \Delta$ y $\gamma \in Q^*$ tal que:

- $w_1 = a \cdot w_2$
- $\gamma_1 = \alpha \cdot \gamma$
- $\gamma_2 = \beta \cdot \gamma$.

Se define $\vdash_{\mathcal{D}}^*$ como la clausura **refleja** y **transitiva** de $\vdash_{\mathcal{D}}$.

Lenguajes de un autómata apilador alternativo

Sea $\mathcal{D} = (Q, \Sigma, \Delta, q_0, F)$ un PDA alternativo y $w \in \Sigma^*$.

Definiciones

- $\mathcal{D}$ **acepta** w si, y solo si, $(q_0, w) \vdash_{\mathcal{D}}^{*} (q_f, \epsilon)$ para algún $q_f \in F$.
- El **lenguaje** aceptado por $\mathcal{D}$ se define como:

$$\mathcal{L}(\mathcal{D}) \;=\; \{w \in \Sigma^* \mid \mathcal{D} \text{ acepta } w\}$$

Lenguajes de un autómata apilador alternativo

Ejemplo

$$\mathcal{D} \quad = \quad (Q, \{a, b\}, \Delta, q_0, F)$$

- $Q = \{q_0, q_1, q_2, q_f\}$
- Δ :

$$\begin{array}{ll}
(q_0, a, q_1 q_0) & q_0 \xrightarrow{a} q_1 q_0 \\
(q_1, a, q_1 q_1) & q_1 \xrightarrow{a} q_1 q_1 \\
(q_1, b, q_2) & q_1 \xrightarrow{a} q_2 \\
(q_2 q_1, b, q_2) & q_2 q_1 \xrightarrow{b} q_2 \\
(q_2 q_0, \epsilon, q_f) & q_2 q_0 \xrightarrow{\epsilon} q_f
\end{array}$$

q_0	
$q_1\ q_0$	a
$q_1\ q_1\ q_0$	a
$q_1\ q_1\ q_1\ q_0$	a
$q_2\ q_1\ q_1\ q_0$	b
$q_2\ q_1\ q_0$	b
$q_2\ q_0$	b
q_f	ϵ

$$\mathcal{L}(\mathcal{D}) \quad = \quad \{a^n b^n \mid n > 0\}$$

Equivalencia entre modelos

Teorema

Para todo autómata apilador $\mathcal{P}$
existe un autómata apilador alternativo $\mathcal{D}$, y viceversa, tal que:

$$\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{D})$$

Desde ahora, usaremos ambos modelos de manera **equivalente**.

Equivalencia entre modelos

Demostración: de $\mathcal{P}$ a $\mathcal{D}$

Sea $\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \bot, F)$ un PDA.

Construimos un **PDA alternativo** $\mathcal{D}_\mathcal{P} = (Q', \Sigma, \Delta', q_0', F')$ tal que:

- $Q' = Q \cup \Gamma \cup \{q_0'\}$
- $F' = F$
- $\Delta' = \{ (q_0', \epsilon, q_0 \bot) \} \cup \{ (pA, a, q\gamma) \mid (p, a, A, q, \gamma) \in \Delta \}$

Ejercicio: demuestre que $\mathcal{L}(\mathcal{P}) = \mathcal{L}(\mathcal{D}_\mathcal{P})$.

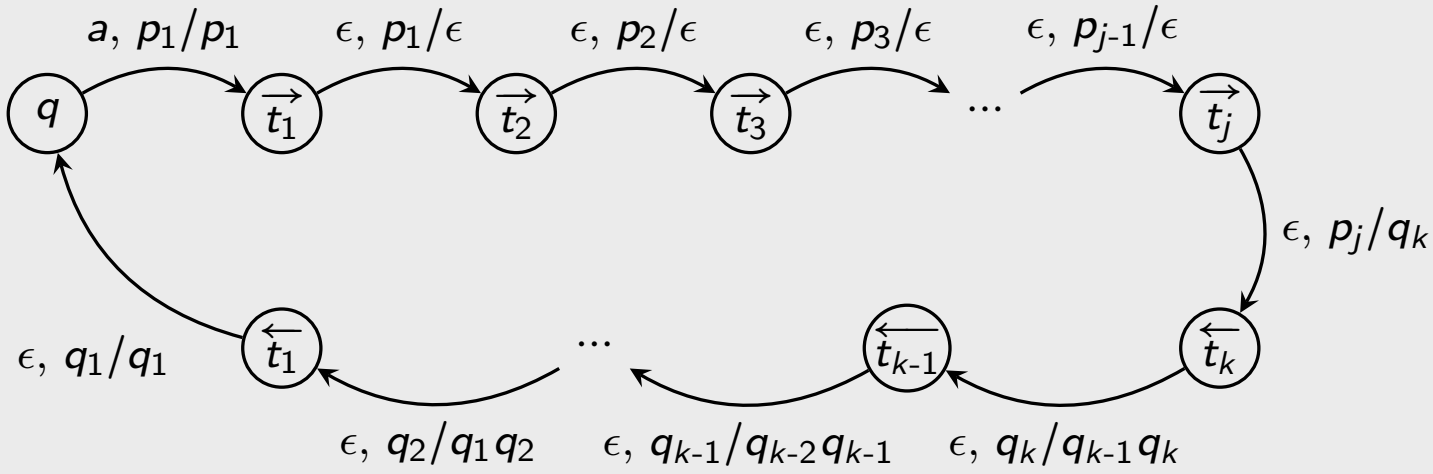
Equivalencia entre modelos

Demostración: de $\mathcal{D}$ a $\mathcal{P}$

Sea $\mathcal{D} = (Q, \Sigma, \Delta, q_0, F)$ un PDA alternativo.

Construimos un **PDA** $\mathcal{P}_\mathcal{D} = (Q', \Sigma, \Gamma', \Delta', q_0', \perp', F')$ tal que ...

... para cada $t = (p_1 \dots p_j, a, q_1 \dots q_k) \in \Delta$, haremos lo siguiente:



Equivalencia entre modelos

Demostración: de $\mathcal{D}$ a $\mathcal{P}$

Sea $\mathcal{D} = (Q, \Sigma, \Delta, q_0, F)$ un PDA alternativo.

Construimos un **PDA** $\mathcal{P}_{\mathcal{D}} = (Q', \Sigma, \Gamma', \Delta', q_0', \perp', F')$ tal que:

- $Q' = \{q, q_f\} \cup \bigcup_{t:(\alpha,a,\beta)\in\Delta} \{\overrightarrow{t_i} \mid 1 \leq i \leq |\alpha|\} \cup \{\overleftarrow{t_i} \mid 1 \leq i \leq |\beta|\}$
- $\Gamma' = Q$
- $\perp' = q_0$
- $q_0' = q$
- $F = \{q_f\}$

Equivalencia entre modelos

Demostración: de $\mathcal{D}$ a $\mathcal{P}$

Sea $\mathcal{D} = (Q, \Sigma, \Delta, q_0, F)$ un PDA alternativo.

Construimos un **PDA** $\mathcal{P}_\mathcal{D} = (Q', \Sigma, \Gamma', \Delta', q_0', \perp', F')$ tal que:

- Δ' : para cada $t = (p_1 \dots p_j, a, q_1 \dots q_k) \in \Delta$ tenemos:

$$(q, a, p_1, \overrightarrow{t_1}, p_1) \in \Delta'$$

$$(\overrightarrow{t_i}, \epsilon, p_i, \overrightarrow{t_{i+1}}, \epsilon) \in \Delta' \qquad \text{para todo } 1 \leq i < j$$

si $k = 0$: $(\overrightarrow{t_j}, \epsilon, p_j, q, \epsilon) \in \Delta'$

si $k > 0$: $(\overrightarrow{t_j}, \epsilon, p_j, \overleftarrow{t_k}, q_k) \in \Delta'$

$$(\overleftarrow{t_i}, \epsilon, q_i, \overleftarrow{t_{i-1}}, q_{i-1}q_i) \in \Delta' \qquad \text{para todo } 1 < i \leq k$$

$$(\overleftarrow{t_1}, \epsilon, q_1, q, q_1) \in \Delta'$$

para todo $p \in F$: $(q, \epsilon, p, q_f, \epsilon) \in \Delta'$

Ejercicio: demuestre que $\mathcal{L}(\mathcal{D}) = \mathcal{L}(\mathcal{P}_\mathcal{D})$.

Outline

Apiladores alternativos (anterior)

Desde CFG a PDA

¿en qué se parecen CFG a PDA?

Teorema

Todo **lenguaje libre de contexto** puede ser descrito equivalentemente por:

- Una gramática libre de contexto (**CFG**).
- Un autómata apilador (**PDA**).

Desde CFG a un PDA

Teorema

Para toda gramática libre de contexto $\mathcal{G}$,
existe un **autómata apilador alternativo** $\mathcal{D}$:

$$\mathcal{L}(\mathcal{G}) = \mathcal{L}(\mathcal{D})$$

¿cómo **construimos** $\mathcal{D}$ desde $\mathcal{G}$?

CFG → PDA: Construcción

Sea $\mathcal{G} = (V, \Sigma, P, S)$ una CFG.

Construimos un **PDA alternativo** $\mathcal{D}$ que acepta $\mathcal{L}(\mathcal{G})$:

$$\mathcal{D} = \big(V \cup \Sigma \cup \{q_0, q_f\}, \Sigma, \Delta, q_0, \{q_f\}\big)$$

La relación de transición Δ se define como:

$$\begin{aligned} \Delta = & \{ (q_0, \epsilon, S \cdot q_f) \} && \cup \\ & \{ (X, \epsilon, \gamma) \mid X \to \gamma \in P \} && \cup \quad (\textbf{Expandir}) \\ & \{ (a, a, \epsilon) \mid a \in \Sigma \} && \quad (\textbf{Reducir}) \end{aligned}$$

CFG → PDA: Construcción

$$
\begin{aligned}
\Delta \quad = \quad & \{ (q_0, \epsilon, S \cdot q_f) \} && \cup \\
& \{ (X, \epsilon, \gamma) \mid X \to \gamma \in P \} && \cup \quad \textbf{(Expandir)} \\
& \{ (a, a, \epsilon) \mid a \in \Sigma \} && \textbf{(Reducir)}
\end{aligned}
$$

Ejemplo: $S \to SS \mid aSb \mid \epsilon$

$$\mathcal{D} \quad = \quad (Q, \{a, b\}, \Delta, q_0, \{q_f\})$$

- $Q = \{S, a, b, q_0, q_f\}$
- $\Delta = \{ \; (q_0, \epsilon, Sq_f), \; (S, \epsilon, SS), (S, \epsilon, aSb), (S, \epsilon, \epsilon), \; (a, a, \epsilon), (b, b, \epsilon) \; \}$

¿qué esta haciendo el autómata apilador $\mathcal{D}$?

CFG $\rightarrow$ PDA: Demostración

Por demostrar:

$$\mathcal{L}(\mathcal{G}) \;=\; \mathcal{L}(\mathcal{D})$$

Dos direcciones:

1. $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$
2. $\mathcal{L}(\mathcal{D}) \subseteq \mathcal{L}(\mathcal{G})$

CFG $\rightarrow$ PDA: Demostración $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$

Para cada $w \in \mathcal{L}(\mathcal{G})$ debemos encontrar
una ejecución de aceptación de $\mathcal{D}$ sobre w.

¿cómo encontramos una ejecución sobre w?

Idea

Para cada árbol de derivación $\mathcal{T}$ de $\mathcal{G}$ sobre w, construimos una ejecución de $\mathcal{D}$ sobre w que recorre el árbol $\mathcal{T}$ **en profundidad**.

Inducción sobre la altura del árbol $\mathcal{T}$

CFG $\rightarrow$ PDA: Demostración $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$

Hipótesis de inducción

Para todo árbol de derivación $\mathcal{T}$ de $\mathcal{G}$ con **altura** h tal que:

- la raíz de $\mathcal{T}$ es X, y
- $\mathcal{T}$ produce la palabra w

entonces $(X \cdot \gamma, w) \vdash_{\mathcal{D}}^{*} (\gamma, \epsilon)$ para todo $\gamma \in Q^{+}$.

Si demostramos esta hipótesis habremos demostrado que $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$

¿por qué?

CFG → PDA: Demostración $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$

Caso base: $h = 1$

Si $\mathcal{T}$ tiene altura 1, entonces:

- $\mathcal{T}$ produce la palabra $w = c$ para algún $c \in \Sigma \cup \{\epsilon\}$ y
- $\mathcal{T}$ consiste de un nodo X y un hijo c con $X \to c$.

Entonces para todo $\gamma \in Q^+$:

$$(X \cdot \gamma, c) \vdash_{\mathcal{D}} (c \cdot \gamma, c) \vdash_{\mathcal{D}}^{*} (\gamma, \epsilon)$$

es una ejecución de $\mathcal{D}$ sobre $w = c$.



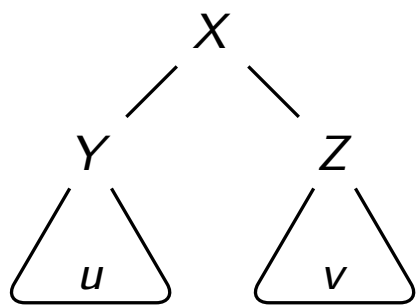
CFG $\rightarrow$ PDA: Demostración $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$

Caso inductivo: $h = n$

Suponemos que el árbol de derivación $\mathcal{T}$ de $\mathcal{G}$ tiene **altura n** tal que:

- la raíz de $\mathcal{T}$ es X, y
- $\mathcal{T}$ produce la palabra w.

Sin perdida de generalidad suponga que $\mathcal{T}$ es de la forma:

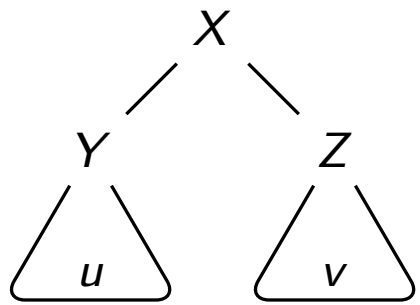


donde $w = u \cdot v$ y $X \rightarrow YZ$.

CFG $\rightarrow$ PDA: Demostración $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$

Caso inductivo: $h = n$

Sin perdida de generalidad suponga que $\mathcal{T}$ es de la forma:



donde $w = u \cdot v$ y $X \rightarrow YZ$.

Por HI, se tiene que para todo $\gamma_1, \gamma_2 \in Q^+$:

$$(Y \cdot \gamma_1, u) \vdash_{\mathcal{D}}^{*} (\gamma_1, \epsilon)$$

$$(Z \cdot \gamma_2, v) \vdash_{\mathcal{D}}^{*} (\gamma_2, \epsilon)$$

CFG $\rightarrow$ PDA: Demostración $\mathcal{L}(\mathcal{G}) \subseteq \mathcal{L}(\mathcal{D})$

Caso inductivo: $h = n$

Por HI, se tiene que para todo $\gamma_1, \gamma_2 \in Q^+$:

$$(Y \cdot \gamma_1, u) \vdash_{\mathcal{D}}^{*} (\gamma_1, \epsilon)$$

$$(Z \cdot \gamma_2, v) \vdash_{\mathcal{D}}^{*} (\gamma_2, \epsilon)$$

Para $\gamma \in Q^+$ **construimos** la siguiente ejecución de $\mathcal{D}$ sobre $w = uv$:

$$(X \cdot \gamma, uv) \vdash_{\mathcal{D}} (YZ \cdot \gamma, uv) \vdash_{\mathcal{D}}^{*} (Z \cdot \gamma, v) \vdash_{\mathcal{D}}^{*} (\gamma, \epsilon)$$



CFG $\rightarrow$ PDA: Demostración $\mathcal{L}(\mathcal{D}) \subseteq \mathcal{L}(\mathcal{G})$

Para cada $w \in \mathcal{L}(\mathcal{D})$ debemos encontrar
un árbol de derivacíon $\mathcal{G}$ para w.

¿cómo encontramos un árbol de derivación para w?

Idea

Si tenemos una ejecución de $\mathcal{D}$ sobre w de la forma:

$$(X \cdot q_f, w) \vdash_{\mathcal{D}}^{*} (q_f, \epsilon)$$

entonces $X \overset{\star}{\underset{\mathcal{G}}{\Rightarrow}} w$

Inducción en la cantidad de pasos de la ejecución.

CFG $\rightarrow$ PDA: Demostración $\mathcal{L}(\mathcal{D}) \subseteq \mathcal{L}(\mathcal{G})$

Hipótesis de inducción

Para toda ejecución de $\mathcal{D}$ sobre w de largo k de la forma:

$$(X \cdot q_f, w) = (\gamma_0, w_0) \vdash_{\mathcal{D}} (\gamma_1, w_1) \vdash_{\mathcal{D}} \cdots \vdash_{\mathcal{D}} (\gamma_k, w_k) = (q_f, \epsilon)$$

entonces $X \overset{\star}{\underset{\mathcal{G}}{\Rightarrow}} w$.

Ejercicio: terminar la demostración.

Cierre de clase

En esta clase vimos:

1. Equivalencia entre gramáticas y apiladores.
2. Apiladores pueden recorrer los árboles de derivación en profundidad.

Próxima clase: Como convertir apiladores en gramáticas.